

CLAW: Learning Continuous Latent Action World Models via Adversarial Latent Regularization

Tewodros W. Ayalew¹, Matthew Jeung^{1,*}, Samuel Wheeler^{3,*}, Xiao Zhang¹,
Andre de la Cruz Arce¹, Kaylene Stocking², Michael Maire¹, Matthew R. Walter²

¹University of Chicago

²Toyota Technological Institute at Chicago

³Argonne National Laboratory

Abstract: We introduce CLAW, a fully end-to-end self-supervised framework for learning a world model jointly with continuous latent action representations directly from action-free videos. Our approach leverages adversarial latent regularization and diffusion-based video generation to capture structured and semantically meaningful action representations while modeling rich, predictive environment dynamics, without relying on any action labels or annotations. By simultaneously training the Latent Action Model and world model, CLAW learns to reason about how inferred actions induce environment transitions from visual observations alone. We show that the resulting latent action world model supports both imitation learning from observation and goal-directed planning. In imitation learning, latent actions extracted from raw videos enable behavior cloning. For planning, CLAW generates sequences of latent actions and maps them to executable actions to reach desired goals. Extensive experiments across diverse tasks and embodiments demonstrate that CLAW produces semantically meaningful latent action representations, supports effective action transfer, and enables planning and imitation from observation, outperforming existing methods.

Keywords: Latent Action Learning, World Models, Latent Action World Model

1 Introduction

Models that predict how actions affect the environment state have long been fundamental to robot planning [1], control [2], and learning [3, 4, 5, 6], enabling robots to evaluate candidate actions, optimize policies in imagination, and generalize to new tasks with improved data efficiency. Inspired by human cognition, renewed attention has been focused on “world models” [7, 8, 9, 10, 11] as forward-predictive models that capture the interaction of robots and other agents with their environment [12, 13, 14, 15, 16, 17, 18]. Contemporary world models often condition their predictions on explicit action labels [15, 19, 20], which limits generalization and transfer between embodiments when actions are agent-specific [21, 22]. Further, action labels limit the potential to scale learning with the large amounts of raw video data available online [23].

Instead, a world model that reasons over learned latent action representations enables generalization across tasks and embodiments, without relying on explicit action labels. Recently methods train such models directly from visual observations alone [24, 25, 26, 21, 23]. However, it remains difficult to learn latent actions that are semantically meaningful, i.e., structured so that transitions with similar effects map to nearby latents across contexts and embodiments. This structure is what ultimately allows the latent actions to be grounded to a robot’s executable actions from only a small amount of supervision [27]. Without care, the latent either *leaks* [28] future-observation content—letting the world model take a shortcut that bypasses learning action semantics—or *collapses* [27, 29] to an uninformative representation [24, 30].

To address these challenges, we present a novel approach for learning continuous latent action representations alongside a world model directly from raw, unlabeled video. Our method leverages a

reciprocal supervision scheme, where the world model guides the Latent Action Model (LAM), which, in turn, informs the world model, resulting in a fully self-supervised, end-to-end training framework for learning latent action world models. We introduce an adversarial latent regularization strategy that promotes semantically meaningful latent actions and counters future-information leakage. By jointly training the LAM and world model, our method simultaneously captures the underlying structure of actions and the ways in which the environment responds. To the best of our knowledge, this work presents the first end-to-end method for jointly learning continuous latent action representations and a world model from video data alone.

Once trained, the learned latent action world model can be used in two complementary modes. First, the model enables the agent to learn policies by imitating behaviors directly from action-free videos as a form of imitation learning from observation (ILfO) setting [31]. Second, it supports planning by generating sequences of latent actions over the learned dynamics to reach specified goals [32, 33].

We evaluate our method across a diverse set of environments, tasks, and embodiments. Quantitatively, we assess planning, latent-action learning from observation, latent policy pretraining, and controllability of the world model. Qualitatively, we analyze the learned latent actions through action transfer [21, 24] and nearest neighbor action retrieval. Overall, we show that CLAW with end-to-end training achieves stronger performance than existing approaches.

2 Related Work

Model-based Learning and World Models Within robotics, world models encompass a broad class of models that predict the consequences of an agent’s actions [14, 34]. Such models are used extensively in model-based reinforcement learning [35, 36, 16, 37, 18, 38] and planning [15, 39, 40, 17]. World models learn dynamics from image-based inputs, which capture fine details and don’t require hand-designed state representations [41, 42, 43, 20, 44, 45]. Typically, these models are trained with ground-truth actions, conditioning future predictions on known control inputs to model environment dynamics and enable downstream planning or policy learning [20, 45, 41, 46].

Latent Action World Models Latent action world models are a kind of action-conditioned world model where the action is inferred from state observations alone, enabling training with raw, unlabeled video and promising transferability across embodiments and contexts [21, 24, 30, 23]. Latent action world models typically use an inverse dynamics model that encodes a latent action representation based on observation sequences, and a forward dynamics model (FDM) that predicts the future state conditioned on the latent action and past observations [25, 47, 48]. Often, latent action world models involve pretraining an action-agnostic FDM and only then introducing inferred actions in a second fine-tuning stage [44, 32, 49, 50, 51]. Instead, we build upon research on ‘action-aware pretraining’ that learns the world model and action representation, which has been shown to improve downstream controllability and transfer [24].

A core challenge in training latent action world models is that the action representation will often embed information about future observations instead of encoding information about the action alone, especially with image-based observations [30]. Prior approaches typically regulate information leakage through explicit bottlenecks, most commonly via quantization [24, 52, 23, 47], variational constraints [21], or capacity-limited continuous latents [53]. Genie [24] adopts VQ-VAE-based discretization to enforce this separation. However, recent work has shown that continuous actions facilitate finer-grained control and often stronger performance than discretization [54, 53, 55]. Other works introduce supervision through a small amount of action-labeled data [30, 53], which reduces leakage but requires some action annotations. In contrast, our approach applies adversarial latent regularization to discourage encoding of future visual information. By jointly training the LAM and world model, we learn representations that capture both action structure and environment dynamics without relying on quantization or action supervision.

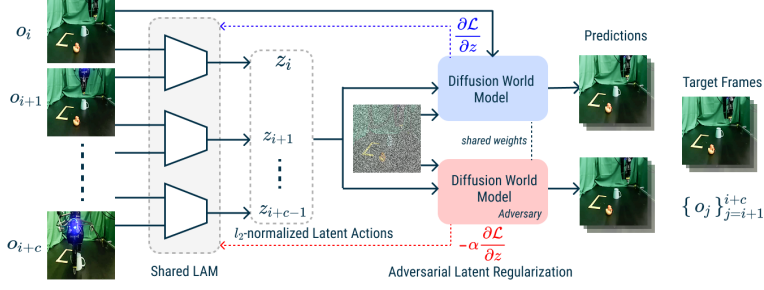


Figure 1: Overview of the CLAW framework. The LAM encodes compact, normalized action representations (left). The adversarial latent regularization employs two diffusion world models with shared weights, with only the first receiving the initial frame alongside the action (right).

3 Latent Action World Model

We formulate the problem as a partially observable Markov decision process (POMDP) denoted as $\langle \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{T}, \mathcal{F} \rangle$, where $\mathcal{S}$ is the latent state space, $\mathcal{O}$ is the observation space, $\mathcal{A}$ is the action space, $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the unknown transition function, and $\mathcal{F} : \mathcal{S} \rightarrow \Delta(\mathcal{O})$ is the unknown emission function. Since s_i is never directly observed, the agent must reason based on the observations.

This world model approximates the observation transition dynamics $P(o_{i+1} | o_i, a_i)$, predicting future observations from past observations and actions. Unlike standard world model formulations, we do not assume access to action annotations or the ability to sample from $\mathcal{A}$ during training. Instead, we introduce a learned latent variable z_i that captures the effect of the underlying action on the observation dynamics. We then model these dynamics as $P(o_{i+1} | o_i, z_i)$, and learn the world model directly from observation sequences. The training data is an action-free video dataset $\mathcal{V} = \{o_{1:T}\}^N$, where each video $o_{1:T} = \{o_1, \dots, o_T\}$ is a sequence of RGB frames $o_i \in \mathbb{R}^{W \times H \times 3}$. Videos vary in length (T) and comprise diverse, unlabeled trajectories. These videos may come, for example, from gameplay recordings or videos of robotic play [56].

Figure 1 provides an overview of our proposed architecture and training setup. Similar to conventional inverse dynamics models (IDMs) [57, 25], the Latent Action Model (LAM) is designed to infer the action responsible for the transition between consecutive observations. We use a ViT backbone [58] to map consecutive observations, o_i, o_{i+1} , into a continuous latent action representation $z_i \in \mathbb{R}^n$, as illustrated in Figure 4(a): $z_i = \text{LAM}_\theta(o_i, o_{i+1})$. The predicted latent actions are constrained to lie on the unit sphere through ℓ_2 normalization.

CLAW subsequently projects the LAM output corresponding to the action token through an MLP to yield the continuous latent action representation, which conditions the world model and captures the action responsible for the observed transition. The LAM is trained end-to-end with the forward dynamics world model, described next.

3.1 Diffusion-based World Model

We represent the dynamics as a diffusion-based world model that takes the initial observation frame o_i and a sequence of latent action embeddings $\{z_j\}_{j=i}^{i+c}$, and predicts the resulting sequence of future frames $\{o_j\}_{j=i+1}^{i+c+1}$. By conditioning on both the initial frame and inferred latent actions, the world model learns to use the inverse dynamics learned by the LAM to produce more accurate predictions. We implement our diffusion-based world model using UViT [59] as the backbone.

The forward diffusion process follows a variance-preserving stochastic differential equation that progressively adds Gaussian noise to the target frames. The diffusion backbone operates on image patches within a transformer architecture. We structure the input as $[t_\tau, \{z_j\}_{j=i}^c, o_i, \{x_j\}_{j=i}^c]$, where t_τ is the diffusion timestep embedding, $\{z_j\}_{j=i}^c \in \mathbb{R}^{c \times d}$ are latent action tokens for c transitions from the LAM, o_i is the set of conditioning frame patches, and x_j are patches from the noised observations

o_j with $i < j \leq i + c$ generated by the forward diffusion process $q(o_t^j \mid o_0^j)$. Latent action tokens are projected to match the transformer embedding dimension.

To maintain causal consistency and prevent information leakage, we use a modified causal attention mask that blocks noisy target tokens from attending to future action tokens or to observations beyond their timestep. As depicted in Figure 4(b), each noised target token attends only to the other tokens in the same frame, to the initial frame tokens, and to the action tokens corresponding to transitions from the initial frame up to the timestep of the noised target frame. This ensures that the model cannot access future information while learning to denoise based on valid action-conditioned dynamics.

Training optimizes a score network $s_\phi(x^t, t, z^t, o_i)$ to predict the noise ϵ added during the forward diffusion process.

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}(0,1), \epsilon \sim \mathcal{N}(0, \mathbf{I})} \left[\|\epsilon - s_\phi(x^t, t, z^t, o_i)\|^2 \right], \quad (1)$$

where the noisy samples x^t are constructed as $x^t = \sqrt{\alpha(t)} x^0 + \sqrt{1 - \alpha(t)} \epsilon$.

At each step, the model conditions on the latent action representation z^t and the reference frame o_i to iteratively denoise the sample. This yields predicted observations that are visually coherent and consistent with the latent action dynamics, enabling accurate and diverse predictions of action-conditioned state transitions.

3.2 Adversarial Information Regularization

The LAM and world model are trained together end-to-end to predict future frames. Unfortunately, this can lead to a degenerate solution in which z_i simply encodes o_{i+1} without learning any useful action representations. To obtain controllable latent actions while preventing information leaking from future observations, we introduce adversarial latent regularization.

Under this approach, we obtain o_{i+1} predictions from the world model on two sets of inputs: one with both o_i and z_i , and one with z_i alone. Without conditioning on o_i , the world model’s predictions should be very poor as long as information about the future observation is not leaking into z_i . Following this intuition, we use a gradient reversal layer inspired by domain adversarial training [60] between the latent action Z_i and the first layer of the world model, only for the inputs without o_i conditioning. During backpropagation, this layer scales and inverts the gradients as $\frac{\partial \mathcal{L}}{\partial z} \leftarrow -\alpha \frac{\partial \mathcal{L}}{\partial z}$. During end-to-end training with the diffusion objective, the world model predictions on both sets of inputs act on the gradients through the LAM in complementary ways. The prediction based on z_i alone effectively discourages the latent action representation from encoding information about the future visual observation due to the gradient reversal layer which penalizes accurate predictions. Meanwhile, the prediction based on both z_i and o_i encourages the LAM to encode useful information about the true underlying action.

During training, the strength of the adversarial signal is controlled by a gradually increasing weight δ , following a sigmoidal schedule

$$\delta = \min \left(\frac{2}{1 + \exp(-\gamma p)} - 1, 0.999 \right) \quad \text{where} \quad p = \min \left(1.0, \frac{\text{step}}{n_{\text{steps}}} \right), \quad (2)$$

where $\gamma = 10$ for all experiments and n_{steps} denotes the total number of training steps. This scheduling strategy initially prioritizes accurate predictions, but places increasing pressure on the LAM to avoid encoding information about future observations.

4 Experiments

We evaluate CLAW in terms of (1) the quality and structure of the learned latent action space, and (2) the ability to do goal-directed planning using the latent action world model. Following prior work [21, 24], we collect task-agnostic interaction data where an agent explores the environment without explicit objectives. Across all experiments, we train latent action world models solely on this exploratory data as detailed in Table 6.

4.1 Baselines

We compare CLAW against two recent latent action methods, AdaWorld [21] and LAPO [25]. AdaWorld uses a two stage pipeline where a LAM is pretrained with a β -VAE objective and then frozen to condition a downstream world model during dynamics learning. LAPO instead learns discrete latent actions using a VQ-VAE based vector quantization objective, which are then used for latent action policy pretraining from unlabeled video data. LAPA [23] is a larger scale variant of a similar latent action learning framework that also uses vector quantization to learn discrete latent actions and supports downstream policy finetuning with limited action-labeled data.

4.2 Benchmarks and Environments

We assess the effectiveness of CLAW across 14 tasks in 5 simulated environments, alongside 3 real-world robotic tasks. Further details for each benchmark are provided in Appendix B.

VP² [61] is a widely adopted control-centric benchmark for evaluating visual planning in table-top manipulation settings. We follow the standard evaluation protocol and evaluate on the Robosuite [62] and RoboDesk [63] environments in planning tasks.

OGBench [64] is an offline goal-conditioned RL benchmark of simulated manipulation tasks, where a UR5e robot arm interacts with articulated objects such as drawers and buttons. We evaluate on three downstream tasks: `Open Drawer`, `Close Drawer`, and `Push Button`.

Crafter [65] is a 2D open-world survival environment where the agent navigates a procedurally generated world, managing resources, avoiding adversaries, harvesting materials, and constructing tools. We evaluate CLAW on two tasks: a long-horizon reward-maximization task that requires collecting as much wood as possible within an episode (`Greedy Wood Collection`), and a goal-conditioned task that requires collecting a single unit of wood (`Single Wood Collection`).

Procggen Benchmark [66] is a diverse suite of procedurally generated RL environments that test an agent’s ability to generalize. We generated trajectories and evaluated downstream visual planning in three tasks: `Climber`, `Jumper`, and `Ninja` (outlined in Appendix B.4.1). Planning is performed using a standard MPC-CEM (Cross-Entropy Method) [67] procedure, as described in Appendix A.5.

Real-world Tabletop Manipulation: We evaluate latent action pretraining on a real-world tabletop manipulation benchmark using a 6-DOF UR5 robot. For a learn-from-human-video setting, we train the latent action world model on the Something-Something V2 dataset [68], which contains 220,847 videos of humans interacting with household items. The world model is trained on human video without access to robot actions or demonstrations. Next, for latent action policy pretraining, we collect an action-free interaction dataset of mixed robot play and human play trajectories, summarized in Table 6. Using the pretrained LAM, we infer latent actions from these videos and use them as supervision for pretraining an ACT policy [69]. For downstream task adaptation, we collect 20 teleoperated UR5 demonstrations for each target task as the limited action-labeled data for finetuning.

The benchmark consists of three tasks: `Push Duck`, `Cover Duck`, and a longer-horizon task `Push and Cover Duck`, which requires both pushing and covering in sequence. Consistent with prior work on long-horizon manipulation evaluation and nuanced robot policy evaluation beyond binary success metrics, we measure performance using a staged progress-based completion metric [70], with more details on tasks in Appendix B.5.

4.3 Evaluating the Learned Latent Action Representation

4.3.1 Latent Action Policy Learning

First, we evaluate our learned latent action representations in the Imitation Learning from Observations (ILfO) setting [24], where the agent learns to perform downstream tasks from expert demonstrations without access to the expert’s actions. To enable imitation from action-free videos, we extract latent

Table 1: Results from Latent Action Policy Learning evaluations on Crafter and OGBench.

Method	Crafter			OGBench		
	Wood Harvest (%)	Avg. Wood	Wood Acquire (%)	Open Drawer (%)	Close Drawer (%)	Push Button (%)
Random	25.0	0.28	27.5	2.5	14.5	8.0
LAPO	35.5	1.12	38.0	4.5	39.5	9.5
AdaWorld	43.5	1.60	46.5	7.5	29.0	16.5
CLAW	86.0	4.46	84.0	13.5	42.0	15.0

actions z_i from expert videos using the trained LAM. These latent actions serve as targets for training a behavior cloning policy $\pi(z_i | o_i)$.

During inference, the predicted latent actions must be mapped to the agent’s true action space. We use a small set of reference videos with action annotations and extract latent actions from these reference videos to form a lookup set. At test time, π predicts latent actions, each of which is then matched to its nearest neighbor in the lookup set. The corresponding true action of this nearest latent is executed in the environment. This retrieval-based procedure allows us to evaluate the quality of the learned behavior without training an additional mapping from latent actions to ground-truth actions.

Results As shown in Table 1, the CLAW-based latent action policy consistently outperforms all baselines across both Crafter tasks, achieving higher success rates and collecting significantly more wood on average in the Greedy Wood Collection task. These results suggest that CLAW’s latent representations more faithfully capture environment interaction dynamics and transfer effectively to downstream policy learning. Notably, despite being pretrained only on general survival videos without explicit supervision for wood harvesting, the learned latent actions still enable effective wood collection, indicating strong disentanglement of action representations from task-specific rewards while preserving meaningful signals for imitation from videos.

4.3.2 Real-world Latent Action Pretraining

Following the experimental protocol of Ye et al. [23], we evaluate whether learned latent actions can be transferred for downstream real-world robot policy learning. Specifically, we study whether pretraining with latent actions instead of ground-truth ones can still improve downstream manipulation performance and sample efficiency.

For this setting, we first train a latent action world model on large-scale human video data. Using the pretrained LAM component, we infer latent actions from action-free expert demonstration videos. These inferred latent actions are then used as supervision to pretrain an ACT robot policy [69]. The policy learns to predict latent actions directly from visual observations.

After pretraining, we freeze the ACT vision encoder and finetune the remaining policy components on downstream manipulation tasks using a small amount of action-labeled robot demonstrations. We compare against training the same ACT architecture from scratch (Scratch) using only ground-truth low-level robot actions and no latent action pretraining. This comparison isolates the effect of latent action pretraining and evaluates whether latent actions inferred by the LAM can serve as an effective alternative supervisory signal for learning transferable visual priors for downstream robot control.

Results Table 3 compares different latent action pretraining methods on the real-world tabletop manipulation benchmark. All approaches substantially outperform training from scratch, highlighting

Table 2: Visual planning evaluations on Procgen.

Method	Crafter	Jumper	Climber	Ninja
LAPO	46.67%	58.33%	53.33%	48.33%
AdaWorld	53.33%	56.67%	66.67%	46.67%
CLAW	68.33%	63.33%	60.00%	56.67%

Table 3: Real-world robotic experiment results.

Method	Push	Cover	Push-Cover
Scratch	0.5	0.4	0.4
LAPA	0.7	0.5	1.3
AdaWorld	0.6	0.6	1.5
CLAW	0.6	0.7	1.7

the effectiveness of latent action supervision for downstream policy learning. Among the latent action pretraining methods, CLAW achieves the strongest overall performance on the long-horizon `Push` and `Cover Duck` task, suggesting improved modeling of compositional behaviors.

4.3.3 Closest Action Retrieval

We qualitatively evaluate the structure of the learned latent action space via L_2 nearest-neighbor retrieval. For each query, we sample a pair of consecutive frames representing an action transition and encode it using the LAM. We then retrieve the n nearest latent actions from the dataset under the L_2 metric and visualize the corresponding frame transitions. Qualitative inspection of the retrieved neighbors allows us to assess whether the latent space organizes semantically similar actions into compact, consistent neighborhoods, reflecting the degree to which the learned representations capture meaningful behavioral structure rather than low-level visual similarity.

Results As shown in Figure 2, across queries, the nearest neighbors in CLAW’s learned latent action space exhibit similar end-effector motion, whereas non-action information like object identity varies substantially. This is in contrast to baseline methods, where nearest neighbor retrieval reveals that the inferred latent actions confuse end effector motion with other non-action scene properties. See Appendix C.2 for further explanation and more examples in Robosuite and other environments.

4.4 Evaluating the Dynamics Model

4.4.1 Visual Planning

Here, we use standard model predictive control (MPC) to plan sequences of latent actions through the world model toward a goal observation. Each predicted latent action is mapped to a true action by retrieving its nearest neighbor from a small set of annotated reference videos. We evaluate on goal-reaching success when executing the plan via the nearest-neighbor actions.

Results Table 4 reports visual planning success rates on the VP² benchmark across Robosuite and RoboDesk tasks. CLAW achieves the highest success rate on four of the six tasks. However, CLAW has lower success rates on the Green and Blue Button tasks. We hypothesize that these tasks are inherently challenging for latent action planning because the goal region is partially occluded and the fixed camera viewpoint makes it difficult to estimate vertical end-effector motion.

Table 2 presents visual planning success rates on Crafter and three Progen tasks using the MPC-CEM planning procedure. CLAW achieves the best performance on three of the four tasks and attains the highest average success rate overall. The consistent improvements over LAPO highlight the advantage of conditioning the world model on expressive continuous latent actions. These results suggest that the structured latent action representation learned by CLAW supports effective planning across diverse tasks and environments. Additional visual planning trajectories are in Appendix C.4.

4.4.2 Action Transfer

We qualitatively evaluate the ability of CLAW to transfer actions across videos using an action swap task, following the protocol from [21]. In this experiment, a sequence of latent actions is extracted from a source video and applied to an initial observation from a different target video. To assess the quality of action transfer, we compare three sequences: the original source and target videos, and the new video with the source video actions applied to the target video initial observation. This comparison provides insight into whether the transferred actions induce the intended dynamics while preserving consistency with the target scene, highlighting the generalizability and robustness of the learned latent action representation.

Results As shown in Figure 3, by $t = 4$, the transferred end-effector motion in the predicted rollout closely matches the original source behavior (see end-effector optical flow in rows three and five). This result is consistent with a learned latent action space that aligns with control in absolute joint angle space. Unlike the case for the end-effector, in the new rollout the motion of the objects

Table 4: Visual planning evaluation results on Robosuite and RoboDesk tasks (mean $\pm$ standard error of the mean (SEM) over three control runs).

Method	Push	Open Slide	Blue Button	Green Button	Red Button	Upright Block
Random	0.0 $\pm$ 0.0%	0.0 $\pm$ 0.0%	10.0 $\pm$ 2.72%	1.11 $\pm$ 1.57%	1.11 $\pm$ 1.57%	0.0 $\pm$ 0.0%
LAPO	18.33 $\pm$ 1.89%	11.67 $\pm$ 3.4%	8.89 $\pm$ 2.40%	6.67 $\pm$ 1.83%	3.33 $\pm$ 2.27%	15.56 $\pm$ 4.54%
AdaWorld*	64.6 $\pm$ 1.52%	5.83 $\pm$ 2.85%	29.17 $\pm$ 2.50%	10.83 $\pm$ 2.50%	10.00 $\pm$ 2.36%	5.00 $\pm$ 0.96%
CLAW	63.7 $\pm$ 1.71%	23.33 $\pm$ 5.67%	12.22 $\pm$ 2.40%	11.11 $\pm$ 0.91%	25.56 $\pm$ 0.91%	43.33 $\pm$ 4.71%

* Results reported from the original paper.

differs from the source video because of their different shapes and positions (see the object optical flow in rows four and six). These results indicate that CLAW successfully disentangles actions from non-action observations and learns robust, plausible dynamics conditioned on these actions. Additional Action Transfer experiments in the Progen environment are detailed in Appendix C.3, which show similar results.

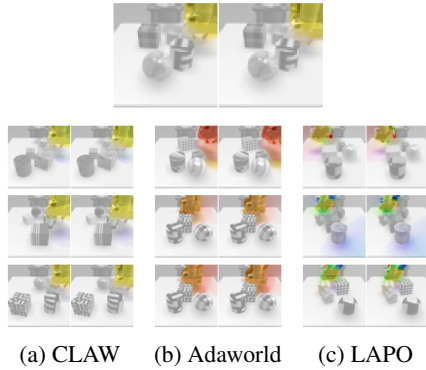


Figure 2: **Closest Latent Action retrieval via L_2 nearest-neighbor:** (Top) query transition. (Bottom) top-3 neighbors retrieved via L_2 search in each method's latent space (CLAW, AdaWorld, LAPO). All frames shown as grayscale with optical flow ($t \rightarrow t+1$) overlaid (Middlebury color wheel).

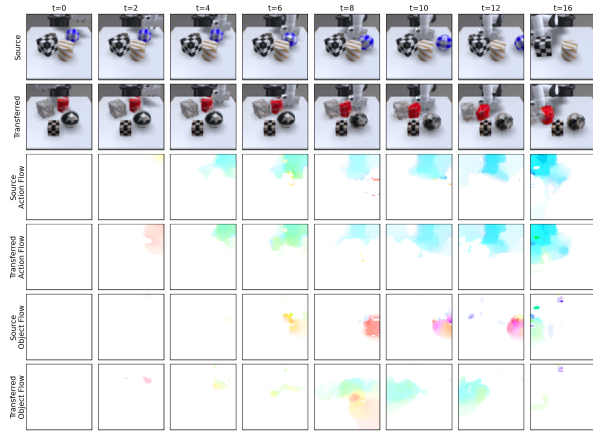


Figure 3: **Latent action transfer via action swap with optical-flow validation.** Row 1: source trajectory. Row 2: transferred rollout from a random target initial frame using source latent actions. Rows 3-4: source and transferred action optical flow. Rows 5-6: source and transferred object flow.

5 Limitations

A key limitation of our latent action world model is that it generates future observations solely based on the most recent frame and latent action, without retaining memory of past generations. This can lead to inaccuracies in partially observable environments. Incorporating explicit memory could improve long horizon consistency and is an important direction for future work. Another unavoidable limitation of latent action world models is the need for a small set of labeled ground-truth action data to map inferred latent actions to controls that can be used in the real world.

6 Conclusion

In this work, we introduce an end-to-end self-supervised method for jointly learning a world model alongside continuous latent action representations, entirely from unlabeled video. We address the future challenge of information leakage with an adversarial regularization strategy, which allows our model to encode meaningful action representations. The learned world model performs competitively with baselines in visual imitation learning, visual planning, and controllability benchmark tasks, demonstrating our approach's effectiveness and ability to transfer across various environments.

References

- [1] D. P. Bertsekas. *Dynamic Programming and Optimal Control: Volume I*. Athena Scientific, 2012.
- [2] Y. Tassa, T. Erez, and E. Todorov. Synthesis and stabilization of complex behaviors through online trajectory optimization. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4906–4913, 2012.
- [3] R. S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Machine Learning Proceedings*, pages 216–224. 1990.
- [4] R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.
- [5] C. G. Atkeson and J. C. Santamaría. A comparison of direct and model-based reinforcement learning. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3557–3564, 1997.
- [6] M. P. Deisenroth and C. E. Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2011.
- [7] G. A. Miller, G. Eugene, and K. H. Pribram. Plans and the structure of behaviour. In *Systems Research for Behavioral Science*, pages 369–382. Routledge, 2017.
- [8] R. C. Conant and W. Ross Ashby. Every good regulator of a system must be a model of that system. *International Journal of Systems Science*, 1(2):89–97, 1970.
- [9] J. Richalet, A. Rault, J.-L. Testud, and J. Papon. Model predictive heuristic control. *Automatica*, 14(5):413–428, 1978.
- [10] T. Lozano-Perez. Robot programming. *Proceedings of the IEEE*, 71(7):821–841, 1983.
- [11] A. E. Bryson. *Applied optimal control: Optimization, estimation and control*. Routledge, 2018.
- [12] A. Clark. Whatever next? Predictive brains, situated agents, and the future of cognitive science. *Behavioral and Brain Sciences*, 36(3):181–204, 2013.
- [13] A. Bubic, D. Y. Von Cramon, and R. I. Schubotz. Prediction, cognition and the brain. *Frontiers in Human Neuroscience*, 4, 2010.
- [14] D. Ha and J. Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- [15] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2019.
- [16] D. Hafner, T. P. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- [17] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering Atari, Go, chess and Shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. ISSN 1476-4687.
- [18] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2024.
- [19] A. Hu, L. Russell, H. Yeo, Z. Murez, G. Fedoseev, A. Kendall, J. Shotton, and G. Corrado. GAIA-1: A generative world model for autonomous driving. *arXiv preprint arXiv:2309.17080*, 2023.

- [20] A. Bar, G. Zhou, D. Tran, T. Darrell, and Y. LeCun. Navigation world models. *arXiv preprint arXiv:2412.03572*, 2025.
- [21] S. Gao, S. Zhou, Y. Du, J. Zhang, and C. Gan. AdaWorld: Learning adaptable world models with latent actions. *arXiv preprint arXiv:2503.18938*, 2025.
- [22] Q. Garrido, T. Nagarajan, B. Terver, N. Ballas, Y. LeCun, and M. Rabbat. Learning latent action world models in the wild. *arXiv preprint arXiv:2601.05230*, 2026.
- [23] S. Ye, J. Jang, B. Jeon, S. Joo, J. Yang, B. Peng, A. Mandlekar, R. Tan, Y.-W. Chao, B. Y. Lin, L. Liden, K. Lee, J. Gao, L. Zettlemoyer, D. Fox, and M. Seo. Latent action pretraining from videos. *arXiv preprint arXiv:2410.11758*, 2025.
- [24] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, et al. Genie: Generative interactive environments. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [25] D. Schmidt and M. Jiang. Learning to act without actions. *arXiv preprint arXiv:2312.10812*, 2024.
- [26] X. Chen, J. Guo, T. He, C. Zhang, P. Zhang, D. C. Yang, L. Zhao, and J. Bian. IGOR: Image-goal representations are the atomic control units for foundation models in embodied ai. *arXiv preprint arXiv:2411.00785*, 2024.
- [27] C. Zhang, T. Pearce, P. Zhang, K. Wang, X. Chen, W. Shen, L. Zhao, and J. Bian. What do latent action models actually learn? In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 146676–146697, 2026.
- [28] J. M. Lee, T. Cho, L. Zhao, and J. Lee. Why latent actions fail, and how to prevent it. *arXiv preprint arXiv:2605.20223*, 2026.
- [29] Y. Wang, F. Zhang, D.-C. Zhan, L. Zhao, K. Wang, and J. Bian. Co-evolving latent action world models. *arXiv preprint arXiv:2510.26433*, 2025.
- [30] A. Nikulin, I. Zisman, D. Tarasov, N. Lyubaykin, A. Polubarov, I. Kiselev, and V. Kurenkov. Latent action learning requires supervision in the presence of distractors. *arXiv preprint arXiv:2502.00379*, 2025.
- [31] F. Torabi, G. Warnell, and P. Stone. Recent advances in imitation learning from observation. *arXiv preprint arXiv:1905.13566*, 2019.
- [32] R. Mendonca, S. Bahl, and D. Pathak. Structured world models from human videos. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [33] F. Ebert, C. Finn, S. Dasari, A. Xie, A. Lee, and S. Levine. Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. *arXiv preprint arXiv:1812.00568*, 2018.
- [34] B. Hou, G. Li, J. Jia, T. An, X. Guo, S. Leng, H. Geng, Y. Ze, T. Harada, P. Torr, O. Mees, M. Pollefeys, Z. Liu, J. Wu, P. Abbeel, J. Malik, Y. Du, and J. Yang. World model for robot learning: A comprehensive survey. *arXiv preprint arXiv:2605.00080*, 2026.
- [35] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721, 2017.
- [36] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. *arXiv preprint arXiv:1708.02596*, 2017.

- [37] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering Atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2022.
- [38] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. *arXiv preprint arXiv:1906.08253*, 2021.
- [39] R. Sekar, O. Rybkin, K. Daniilidis, P. Abbeel, D. Hafner, and D. Pathak. Planning to explore via self-supervised world models. *arXiv preprint arXiv:2005.05960*, 2020.
- [40] V. Micheli, E. Alonso, and F. Fleuret. Transformers are sample-efficient world models. *arXiv preprint arXiv:2209.00588*, 2023.
- [41] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret. Diffusion for world modeling: Visual details matter in Atari. *arXiv preprint arXiv:2405.12399*, 2024.
- [42] Y. Bai, D. Tran, A. Bar, Y. LeCun, T. Darrell, and J. Malik. Whole-body conditioned egocentric video prediction. *arXiv preprint arXiv:2506.21552*, 2025.
- [43] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, Mojtaba, Komeili, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, S. Arnaud, A. Gejji, A. Martin, F. R. Hogan, D. Dugas, P. Bojanowski, V. Khalidov, P. Labatut, F. Massa, M. Szafraniec, K. Krishnakumar, Y. Li, X. Ma, S. Chandar, F. Meier, Y. LeCun, M. Rabbat, and N. Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [44] J. Wu, S. Yin, N. Feng, X. He, D. Li, J. Hao, and M. Long. iVideoGPT: Interactive videogpts are scalable world models. *arXiv preprint arXiv:2405.15223*, 2024.
- [45] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2025.
- [46] Y. Wang, R. Syed, F. Wu, M. Zhang, A. Onol, J. Barreiros, H. Nayyeri, T. Dear, H. Zhang, and Y. Li. Interactive world simulator for robot policy training and evaluation. *arXiv preprint arXiv:2603.08546*, 2026.
- [47] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li. UniVLA: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- [48] Z. J. Cui, H. Pan, A. Iyer, S. Haldar, and L. Pinto. DynaMo: In-domain dynamics pretraining for visuo-motor control. *arXiv preprint arXiv:2409.12192*, 2024.
- [49] NVIDIA, :, N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding, D. Dworakowski, J. Fan, M. Fenzi, F. Ferroni, S. Fidler, D. Fox, S. Ge, Y. Ge, J. Gu, S. Gururani, E. He, J. Huang, J. Huffman, P. Jannaty, J. Jin, S. W. Kim, G. Klár, G. Lam, S. Lan, L. Leal-Taixe, A. Li, Z. Li, C.-H. Lin, T.-Y. Lin, H. Ling, M.-Y. Liu, X. Liu, A. Luo, Q. Ma, H. Mao, K. Mo, A. Mousavian, S. Nah, S. Niverty, D. Page, D. Paschalidou, Z. Patel, L. Pavao, M. Ramezanali, F. Reda, X. Ren, V. R. N. Sabavat, E. Schmerling, S. Shi, B. Stefaniak, S. Tang, L. Tchapmi, P. Tredak, W.-C. Tseng, J. Varghese, H. Wang, H. Wang, H. Wang, T.-C. Wang, F. Wei, X. Wei, J. Z. Wu, J. Xu, W. Yang, L. Yen-Chen, X. Zeng, Y. Zeng, J. Zhang, Q. Zhang, Y. Zhang, Q. Zhao, and A. Zolkowski. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025.
- [50] H. Che, X. He, Q. Liu, C. Jin, and H. Chen. GameGen-X: Interactive open-world game video generation. *arXiv preprint arXiv:2411.00769*, 2024.
- [51] H. He, Y. Zhang, L. Lin, Z. Xu, and L. Pan. Pre-trained video generative models as world simulators. *arXiv preprint arXiv:2502.07825*, 2025.
- [52] Y. Huang, W. Wan, Y. Yang, C. Callison-Burch, M. Yatskar, and L. Liu. CoMO: Controllable motion generation through language guided pose code editing. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 180–196, 2024.

- [53] J. Yang, Y. Shi, H. Zhu, M. Liu, K. Ma, Y. Wang, G. Wu, T. He, and L. Wang. CoMo: Learning continuous latent motion from internet videos for scalable robot learning. *arXiv preprint arXiv:2505.17006*, 2025.
- [54] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>.
- [55] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success, 2025. URL <https://arxiv.org/abs/2502.19645>.
- [56] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet. Learning latent plans from play. In *Proceedings of the Conference on Robot Learning (CoRL)*, pages 1113–1132, 2020.
- [57] A. Edwards, H. Sahni, Y. Schroecker, and C. Isbell. Imitating latent policies from observation. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 1755–1763, 2019.
- [58] A. Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [59] F. Bao, S. Nie, K. Xue, Y. Cao, C. Li, H. Su, and J. Zhu. All are worth words: A vit backbone for diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22669–22679, 2023.
- [60] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. March, and V. Lempitsky. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35, 2016.
- [61] S. Tian, C. Finn, and J. Wu. A control-centric benchmark for video prediction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [62] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020.
- [63] H. Kannan, D. Hafner, C. Finn, and D. Erhan. RoboDesk: A multi-task reinforcement learning benchmark. <https://github.com/google-research/robodesk>, 2021.
- [64] S. Park, K. Frans, B. Eysenbach, and S. Levine. Ogbench: Benchmarking offline goal-conditioned rl. In *International Conference on Learning Representations*, volume 2025, pages 94937–94982, 2025.
- [65] D. Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021.
- [66] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman. Leveraging procedural generation to benchmark reinforcement learning. *arXiv preprint arXiv:1912.01588*, 2019.
- [67] P.-T. De Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein. A tutorial on the cross-entropy method. *Annals of Operations Research*, 134(1):19–67, 2005.
- [68] R. Goyal, S. Ebrahimi Kahou, V. Michalski, J. Materzynska, S. Westphal, H. Kim, V. Haenel, I. Freund, P. Yianilos, M. Mueller-Freitag, et al. The” something something” video database for learning and evaluating visual common sense. In *Proceedings of the IEEE international conference on computer vision*, pages 5842–5850, 2017.

- [69] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [70] H. Kress-Gazit, K. Hashimoto, N. Kuppuswamy, P. Shah, P. Horgan, G. Richardson, S. Feng, and B. Burchfiel. Robot learning as an empirical science: Best practices for policy evaluation. *arXiv preprint arXiv:2409.09491*, 2024.
- [71] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022.
- [72] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [73] S. Moon, J. Yeom, B. Park, and H. O. Song. Discovering hierarchical achievements in reinforcement learning via contrastive learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 63674–63686, 2023.
- [74] S. Baker, D. Scharstein, J. P. Lewis, S. Roth, M. J. Black, and R. Szeliski. A database and evaluation methodology for optical flow. *International journal of computer vision*, 92(1):1–31, 2011.

A CLAW Implementation Details

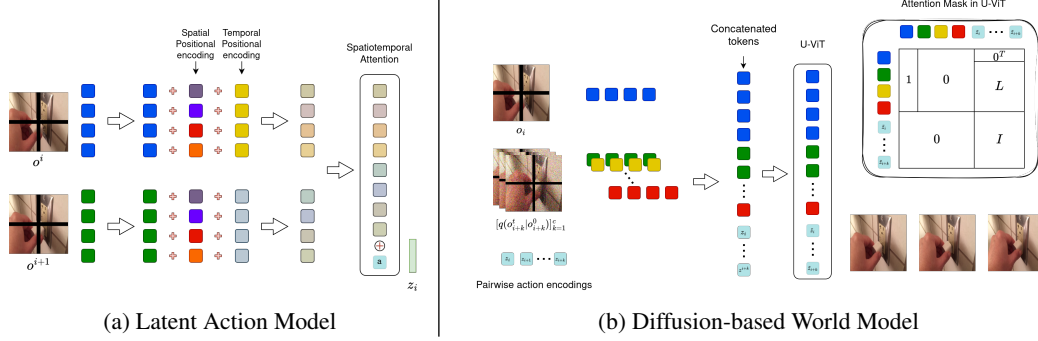


Figure 4: CLAW consists of two main components: a ViT based latent action model (LAM) and a U-ViT based diffusion world model. The LAM is designed to infer compact latent actions directly from visual state transitions, while the diffusion model predicts future observations conditioned on these latent actions.

A.1 Latent Action Model

CLAW’s LAM uses a ViT backbone [58, 71] as a transition model that maps consecutive observations to a continuous action code. Given a pair of frames $\{o_i, o_{i+1}\}$, the encoder produces a latent action $z_i \in \mathbb{R}^n$ as illustrated in Figure 4(a): $z_i = \text{LAM}_\theta(o_i, o_{i+1})$.

These patches are projected into the model’s hidden dimension and augmented with both spatial and temporal positional encodings. The spatial positional encoding is shared across corresponding spatial locations over time, enabling the model to preserve spatial correspondence between frames, while the temporal positional encoding identifies the timestep associated with each frame. Together, these encodings produce a structured spatio temporal token representation that captures both appearance and temporal evolution.

A learnable action token is concatenated with the spatio temporal patch tokens and processed using a spatio temporal Vision Transformer encoder. The full sequence is processed by a ViT-style transformer encoder, which outputs the latent action that summarizes the transition between the two input observations. The final representation of this token is projected into a compact latent action vector, which is subsequently ℓ_2 normalized to produce the latent action embedding. This latent representation serves as a compressed description of the transition between observations and forms the core of the latent action model.

A.2 Diffusion-based World Model

Our world model is a diffusion-based video predictor built on a U-ViT [59] backbone. It learns to denoise target frames conditioned on latent actions extracted from observed transitions, so rollouts proceed by applying a sequence of inferred actions while iteratively refining predictions from an initial scene. Because generation is autoregressive in time, the model must respect temporal causality: predictions at step t may use the starting observation and actions up to t , but must not depend on future frames or future actions. We therefore train with a modified causal attention mask that enforces this structure in the token sequence rather than relying on the network to learn it implicitly.

The mask is designed to block information leakage at the token level. Noisy target tokens cannot attend to future action tokens, nor to visual tokens from timesteps beyond their own. As illustrated in Figure 4(b), each noised target token is allowed to attend only to (i) other tokens within the same frame, (ii) tokens from the initial conditioning frame, and (iii) action tokens associated with transitions from the initial frame through the timestep of the noised target. Intuitively, a token being

Table 5: Training and model configuration for the latent action model and diffusion world model.

Component	Configuration
Training Steps	100,000
Batch Size	512
Optimizer	AdamW
Learning Rate	3×10^{-4}
Weight Decay	0.03
Betas	(0.99, 0.99)
LR Scheduler	Customized
Warmup Steps	5,000
Latent Action Model (LAM)	
Spatio-Temporal Encoder	ViT
Input Resolution	64×64
Patch Size	4×4
Embedding Dimension	512
Depth	8
Attention Heads	8
Latent Action Dimension	32
Latent Normalization	ℓ_2 normalization
Diffusion World Model	
Backbone	U-ViT
Embedding Dimension	512
Depth	12
Attention Heads	8
Patch Size	4×4
Diffusion Process	Variance-preserving SDE
β_0	0.1
β_1	20
Conditioning	Query frame + noised target + latent actions
Max Action Tokens	4
Attention Mask	Structured causal masking
Sampler	Probability-flow ODE
Sampling Steps	50

denoised at time t may aggregate spatial context within its frame, retain access to the rollout’s starting state, and read the action history that explains how the scene should evolve up to t , but it cannot “peek” at later actions or later observations. This restricted attention pattern keeps denoising aligned with valid action-conditioned dynamics and makes sequential transfer reliable: each generated frame is produced from causal action context, not from future information in the training or inference sequence.

We model frame generation with a variance-preserving diffusion process. Given a clean target frame o_0 (and conditioning on a query frame and latent action), the forward noising distribution is

$$q(o_t | o_0) = \mathcal{N}(\sqrt{\alpha_t} o_0, \beta_t I), \quad x_t = \sqrt{\alpha_t} o_0 + \sqrt{\beta_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (3)$$

where $t \in [0, 1]$ indexes noise level, $\beta_t = 1 - \alpha_t$.

A.3 CLAW Configuration

Training and hyperparameter details for CLAW are provided in Table 5.

A.4 Latent Action Policy Training Pipeline

A.4.1 Latent Action Extraction

For each partition, latent actions are computed from consecutive pairs of visual observations using the Latent Action Model (LAM). Given two successive frames (o_t, o_{t+1}) , a short temporal sequence

is formed (i.e., the action chunk) and passed through LAM. The LAM produces a latent action embedding $z_t = \text{LAM}(o_t, o_{t+1})$. We use a latent vector size of 32 for all our experiments. These latent vectors represent the transformation between consecutive observations. For each video, we extract latent actions and save them together with the corresponding video frames: $\mathcal{D} = \{(o^i, z_i)\}_{i=1}^T$.

A.4.2 Latent Action Policy Network

The policy network is implemented as a convolutional neural network based on a ResNet-18 [72] backbone. The final fully connected layer is replaced with a linear projection to the latent action dimension 32 to match our latent action dimension. Given an observation o_t , the policy predicts a latent action $\hat{z}_t = \pi_\theta(o^i)$ where π_θ denotes the policy network with parameters θ . The output vector is normalized to unit norm to match the geometry of the latent action space learned by the encoder.

A.4.3 Training Objective

The policy is trained in a supervised regression setting using the extracted latent actions as targets. For a batch of observations and latent labels (o^i, z_t) , the training objective minimizes the L1 distance between predicted and target latent actions:

$$\mathcal{L}(\theta) = \|f_\theta(o_t) - z_t\|_1.$$

The Adam optimizer is used for training with a learning rate of 10^{-4} and a batch size of 32. Training is performed for 100 epochs.

A.5 Visual Planning

We perform planning using Model Predictive Control combined with the Cross Entropy Method (MPC-CEM) [67]. At each planning step, the planner optimizes a sequence of latent actions over a finite horizon using the learned diffusion world model. Given the current observation o_t and a goal observation o_g , the planner samples candidate latent action sequences

$$\mathbf{z}_{1:H} = \{z_1, z_2, \dots, z_H\},$$

where each latent action $z_i \in \mathbb{R}^d$. For each sampled sequence, the world model, f_θ , predicts a future rollout

$$\hat{o}_{t+1:t+H} = f_\theta(o_t, \mathbf{z}_{1:H}).$$

The planning objective minimizes a visual goal reaching cost computed on the final predicted frame:

$$J(\mathbf{z}_{1:H}) = d(\hat{o}_{t+H}, o_g),$$

where $d(\cdot, \cdot)$ denotes a task dependent distance metric between the predicted terminal observation and the goal observation. CEM iteratively updates a Gaussian sampling distribution over latent action sequences using the top performing elite samples. After optimization, the first k latent actions from the best sequence are executed in the environment before replanning. Planning terminates when the goal is reached or a maximum interaction budget is exceeded. The complete MPC-CEM planning procedure is summarized in Algorithm 1.

For the Procgen and Crafter planning experiments, we employ model MPC-CEM as the trajectory optimizer. The planning objective is defined using a pixel-wise mean squared error between predicted observations and the goal image. At each MPC iteration, we run five CEM updates, where 64 candidate action sequences of horizon length five are sampled and evaluated using the learned world model. The top 32 sequences, ranked by the visual cost, are used to update the sampling distribution. After optimization, the first five actions of the best sequence are executed in the environment before replanning. This process continues for up to 20 MPC iterations or until task success is achieved. CLAW rollouts are generated using 50 diffusion denoising steps.

For VP², we used the benchmark’s provided planner, which scores sampled action sequences via a cost function combining pixel-wise MSE to the goal image and a learned task-success classifier. We used the benchmark’s recommended weights of $\lambda_1 = 0.5$ and $\lambda_2 = 10$.

Algorithm 1 Model-Based Visual Planning with MPC and CEM

Require: World model f_θ , environment $\mathcal{E}$, initial observation o_0 , goal observation o_g , planning horizon H , replan interval k , CEM iterations I , number of samples N , number of elite samples K

Ensure: Executed latent actions $\mathcal{Z}_{exec}$

```
1:  $o_t \leftarrow o_0$ 
2:  $\mathcal{Z}_{exec} \leftarrow \emptyset$ 
3:  $t \leftarrow 0$ 
4: while not success and  $t < t_{\max}$  do
5:    $\mu \leftarrow \mathbf{0}$ 
6:    $\Sigma \leftarrow \sigma_0^2 I$ 
7:   for  $j = 1$  to  $I$  do
8:     Sample latent action sequences
                                     
$$\{\mathbf{z}_{1:H}^{(i)}\}_{i=1}^N \sim \mathcal{N}(\mu, \Sigma)$$

9:     for each  $\mathbf{z}_{1:H}^{(i)}$  do
10:      Predict future rollout
                                     
$$\hat{o}_{t+1:t+H}^{(i)} = f_\theta(o_t, \mathbf{z}_{1:H}^{(i)})$$

11:      Compute planning cost
                                     
$$J_i = d(\hat{o}_{t+H}^{(i)}, o_g)$$

12:    end for
13:    Select top- $K$  latent sequences:  $\mathcal{Z}_{elite}$ 
14:    Update sampling distribution
                                     
$$\mu \leftarrow \text{mean}(\mathcal{Z}_{elite})$$

                                     
$$\Sigma \leftarrow \text{cov}(\mathcal{Z}_{elite})$$

15:  end for
16:  Select best latent plan
                                     
$$\mathbf{z}_{1:H}^* = \arg \min_{\mathbf{z}_{1:H}^{(i)}} J_i$$

17:  Execute first  $k$  latent actions
                                     
$$\mathbf{z}_{1:k}^{exec} = \mathbf{z}_{1:k}^*$$

18:  Step environment
                                     
$$o_{t+k} = \mathcal{E}(o_t, \mathbf{z}_{1:k}^{exec})$$

19:   $\mathcal{Z}_{exec} \leftarrow \mathcal{Z}_{exec} \cup \mathbf{z}_{1:k}^{exec}$ 
20:   $o_t \leftarrow o_{t+k}$ 
21:   $t \leftarrow t + k$ 
22:  success  $\leftarrow d(o_t, o_g) < \epsilon$ 
23: end while
24: return  $\mathcal{Z}_{exec}$ 
```

B Benchmark and Dataset

In this Appendix section, we give further details about each benchmark experiment and the corresponding tasks. See Table 6 for an overview and Figure 5 for visuals.



Figure 5: Benchmarks and environments that we evaluate CLAW on. Top row (left-to-right): VP2 Robosuite, VP2 RoboDesk, OGBench Scene, Real-world human, Real-world robot. Bottom row (left-to-right): Procgen Climber, Procgen Ninja, Procgen Jumper, Crafter, Something-Something v2.

Table 6: Breakdown of CLAW benchmarks: environments, latent action world model (LAWM) training details, task adaptation details, and evaluation details. For evaluation methods, we abbreviate Closest Action Retrieval (AR), Action Transfer (AT), Visual Planning (VP), Latent Action Policy Learning (PL), Latent Action Pretraining (LAP), and Controllability (C).

Env	LAWM Training		Task Adaptation		Evaluation	
	Dataset	# Frames	Method	# Trajs	Methods	# Tasks
VP ²	Robosuite	175k	Retrieval	40	AR, AT, VP	1
	RoboDesk	13.9k	Retrieval	40/task	AR, AT, VP	5
OGBench	Oracle	826k	Retrieval	25 /task	AR, AT, VP, PL, C	3
Crafter	Human Expert	62k	Retrieval	20	AR, AT, VP, PL, C	2
Procgen	Climber	70k	Retrieval	40	AR, AT, VP	1
	Jumper	70k	Retrieval	40	AR, AT, VP	1
	Ninja	70k	Retrieval	40	AR, AT, VP	1
Real-World	Something v2	168k videos	Fine-tuning	20/task	LAP	3

B.1 VP2

For the VP² [61] benchmark, we train CLAW and the baselines on the benchmark datasets and evaluate their performance on the provided planning tasks without additional task-specific supervision. For Robosuite, we use the 5k video dataset and evaluate on 100 tabletop visual planning tasks derived from Zhu et al. [62]. For RoboDesk [63], we use 13.9k training videos and evaluate on 7 tabletop RoboDesk tasks.

B.2 OGBench

For the OGBench [64] benchmark, we collected 750 videos using the environment’s built-in oracles to train our world models, then evaluated latent action policy learning on three downstream tasks: Open Drawer, Close Drawer, and Push Button.

B.2.1 OGBench Data Generation

We generated data within OGBench’s [64] Scene environment, a manipulation setup with a single robot arm, sliding handle drawer, sliding window, a cube, and two buttons. We employed OGBench’s default PlanOracle, a scripted, non-Markovian expert policy, to collect 750 total trajectories with noise set to zero, filtering the tasks to just ‘drawer’ and ‘button’. For each trajectory, we set a maximum episode length of 1001 (max.episode.length=1001), which terminates early upon

task completion. For the pixel-based observations, we use the untransparent robotic arm setting (`pixeltransparent_arm=False`).

B.3 Crafter

In the Crafter [65] environment, we evaluate the latent action models on two tasks. The `Greedy Wood Collection` task performance is measured by two distinct metrics, the first being success rate, the percentage of trajectories in which the agent collects at least one piece of wood. The second measures the average number of wood units collected, reflecting the agent’s ability to maximize cumulative reward over a long horizon. The `Single Wood Collection` is also measured in success rate. All policy evaluations are performed by deploying the learned model in the environment and measuring its performance over 200 random seeds over the environment state.

B.3.1 Crafter Benchmark Data Generation

For our imitation learning, we generated expert video samples in the Crafter game environment [65] using a PPO agent based on Achievement Distillation [73]. We trained separate agents on two distinct tasks, `Single Wood Collection` and `Greedy Wood Collection`.

Agent Setup and Training Details We use PPO_AD (Achievement Distillation) [73] implemented from the open-sourced codebase, using the default `PPOADModel / PPOADAlgorithm` architecture and losses as defined in the original paper and repository. Unless otherwise noted, we follow the repository defaults, with the following key settings: 8 parallel workers (`nproc=8`), 512-step rollouts (`nstep=512`), discount $\gamma = 0.95$, and GAE $\lambda = 0.65$. PPO optimization uses 3 epochs per update with 8 minibatches (`nepoch=3, nbatch=8`), clipping $\epsilon = 0.2$, value loss coefficient 0.5, entropy coefficient 0.01, learning rate 3×10^{-4} , and gradient clipping 0.5. We terminated training after 80 epochs.

Crafter Environment and Reward We train in Crafter, which provides an environment reward at each timestep, as the sum of two components. First, an *achievement* reward gives +1 only when an achievement is unlocked for the first time within the current episode (and 0 otherwise). The only achievement relevant to our benchmarks is `collect_wood`, which is given upon the first wood collected by the agent. Second, a *health-shaping* reward provides a small dense signal equal to -0.1 for each health point lost and $+0.1$ for each health point regenerated. Beyond these standard rewards, we make the following edits for the two specific tasks:

`Single Wood Collection`: The only achievement that yields a +1 achievement reward is `collect_wood`, with all other achievement rewards removed.

`Greedy Wood Collection`: In addition to removing all non wood-collecting achievement rewards, we add a dense reward of +1 for each additional wood collected beyond the first.

B.4 Procgen Benchmark

For the Procgen [66] benchmark, we followed the data collection procedure of AdaWorld (outlined in Appendix B.4.1). We generated 500 videos for each task (Climber, Jumper, and Ninja) as training data. We evaluate the trained model on downstream visual planning tasks. For each environment, planning problems are constructed by rolling out random action sequences of length 15 and using the initial frame as the starting observation for the world model and the frame at ($t = 15$) as the target. Planning is performed using a standard MPC CEM procedure, as described in Appendix A.5.

B.4.1 Procgen Benchmark Data Generation

We followed the same method as AdaWorld [21] to collect frame transitions from the following Procgen benchmarks tasks: Climber, Jumper, and Ninja. Within each task, we collected 500 videos of an agent acting in the environment, which takes a random action at each step. Each video started from a random scene, and terminated after 1000 environment steps or when the environment returned

done. Following AdaWorld, we replaced uniform action sampling with a biased schedule. For a short window we increased the probability of selecting one action, then switched to another. Cycling the favored action this way encouraged each video to cover a broader range of scenes.

B.5 Real-world

The first two tasks, `Push Duck` and `Cover Duck`, are short-horizon tasks involving pushing and pick-and-place behaviors, while `Push` and `Cover Duck` is a longer-horizon task that requires both behaviors in sequence. Consistent with prior work on long-horizon manipulation evaluation and recent recommendations for more nuanced robot policy evaluation beyond binary success metrics [70], we measure performance using a staged progress-based completion metric. The two short-horizon tasks are treated as single-stage tasks and receive a score of 1 upon successful completion. The long-horizon `Push` and `Cover Duck` task is decomposed into three sequential stages: (1) pushing the duck to a target location, (2) grasping the cup, and (3) placing the cup on top of the duck. The final score therefore ranges from 0 to 3 depending on the number of successfully completed stages. Each task is evaluated over 10 independent trials, and we report the average progress score across runs.

C Additional Experiments and Results

C.1 Controllability

Table 7: PSNR and Δ_t PSNR for prediction fidelity and controllability under random latent actions.

Method	Crafter		Climber		Jumper		Ninja	
	PSNR $\uparrow$	Δ PSNR $\uparrow$	PSNR $\uparrow$	Δ PSNR $\uparrow$	PSNR $\uparrow$	Δ PSNR $\uparrow$	PSNR $\uparrow$	Δ PSNR $\uparrow$
CLAW	8.31	8.90	15.87	6.62	17.38	10.21	20.51	9.30
CLAW (No Adv.)	1.33	0.79	10.74	4.08	12.52	5.52	20.41	7.40

We evaluate the controllability of our method using the Δ_t PSNR metric [24, 21]. This metric measures how strongly the generated rollouts depend on the conditioning actions by comparing video predictions conditioned on latent actions inferred from ground truth transitions against those conditioned on randomly sampled latent actions. A higher Δ_t PSNR indicates stronger action influence and improved controllability, as the world model produces distinct futures in response to different latent action inputs. Results for PSNR and Δ_t PSNR on Crafter and Procgen environments are reported in Table 7.

C.2 Retrieval Task

As shown in Figure 6, our model retrieves a more diverse set of transitions that correspond to the correct actions, while exhibiting reduced reliance on superficial visual attributes such as color or texture. In contrast, AdaWorld’s Latent Action Model frequently retrieves transitions from visually similar scenes, revealing a bias toward scene specific appearance cues rather than action semantics.

In Figure 6, the query corresponds to a `DO` action where the agent interacts with an object in the environment. CLAW retrieves transitions that capture similar object interactions, whereas AdaWorld mainly retrieves transitions from visually similar scenes with limited or no interaction. On the right side of the figure, both CLAW and AdaWorld recover the correct action category; however, CLAW shows substantially less reliance on scene color and retrieves transitions from a more diverse set of environments.

Additionally, Figure 10 presents a query transition corresponding to a `chopping tree` action. The nearest neighbors retrieved by CLAW consistently correspond to the same chopping behavior. Although AdaWorld also retrieves several chopping tree examples, it additionally includes multiple `no-op` actions. In contrast, LAPO retrieves a heterogeneous mixture of actions involving different motions and behaviors. These results indicate that CLAW learns action representations with stronger semantic clustering compared to the baselines.

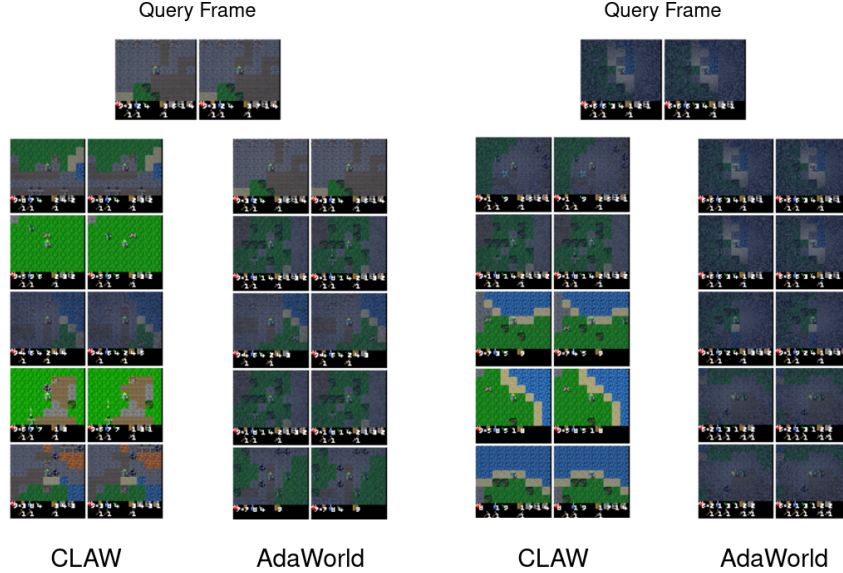


Figure 6: Nearest action retrieval comparison between CLAW and AdaWorld. For each query transition, we retrieve the closest latent actions from the latent action embeddings of the Crafter dataset and visualize their corresponding frame pairs. CLAW retrieves a more diverse set of transitions that accurately reflect the underlying action semantics, with reduced sensitivity to color and texture. In contrast, AdaWorld predominantly retrieves transitions from visually similar scenes, suggesting a stronger dependence on appearance cues rather than action specific dynamics.

Additional 15 closest action retrieval examples for different query transitions, along with a broader set of transitions, are shown in Figures 7, 8, and 9. Overall, these results demonstrate that CLAW’s latent action representations are robust and transferable, enabling more reliable planning and control in downstream visual tasks.

C.3 Action Transfer

In Figure 3 and Figure 7, CLAW’s end-effector motion corresponds to rightward and slightly downward displacement much like the source (yellow in the Middlebury color wheel [74], key in Figure 11), even when object identity and scene layout differ substantially from the query. This consistency is reflected in the optical flow visualizations, where retrieved neighbors share a coherent dominant flow orientation, in contrast to retrievals from baseline methods that exhibit heterogeneous or weakly aligned flow patterns.

Additional Action Transfer experiments in the Procgen Ninja, Climber, and Jumper environments are shown in Figure 12, Figure 13, and Figure 14, respectively. We compare CLAW, AdaWorld, and LAPO in each environment. Across all environments, as predictions progress further away from the initial observation, AdaWorld and LAPO show significantly more degradation than CLAW.

C.4 Visual Planning Task

We display successful and failed visual planning trajectories for the VP2 Robodesk and Robosuite benchmarks in Figure 15 and Figure 16. Visual planning trajectory successes and failures for the Procgen Ninja and Jumper environments are shown in Figure 17 and Figure 18.



Figure 7: **Closest Latent Action retrieval via L_2 nearest-neighbor:** (Top) query transition. (Bottom) top-10 neighbors retrieved via L_2 search in each method’s latent space (CLAW, AdaWorld, LAPO). Retrieval is independent per encoder. All frames shown as grayscale with optical flow ($t \rightarrow t+1$) overlaid (Middlebury color wheel).

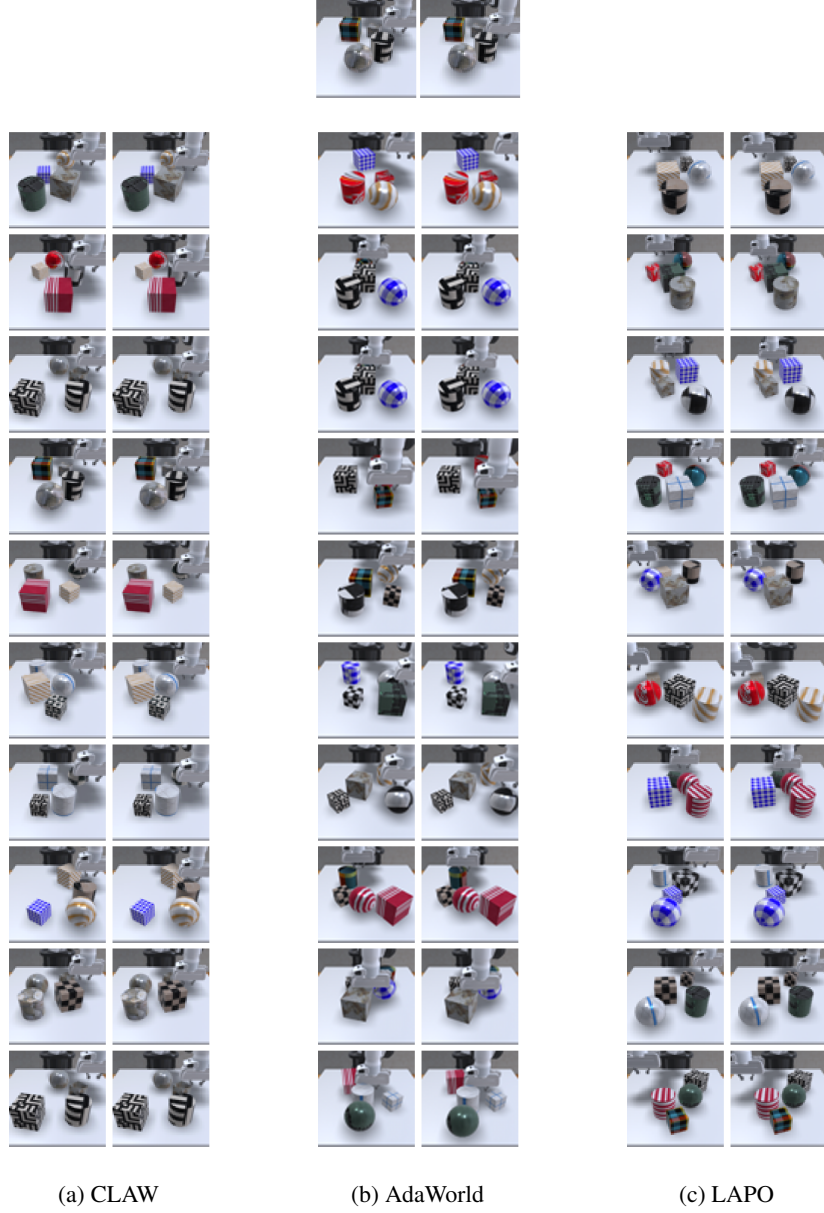
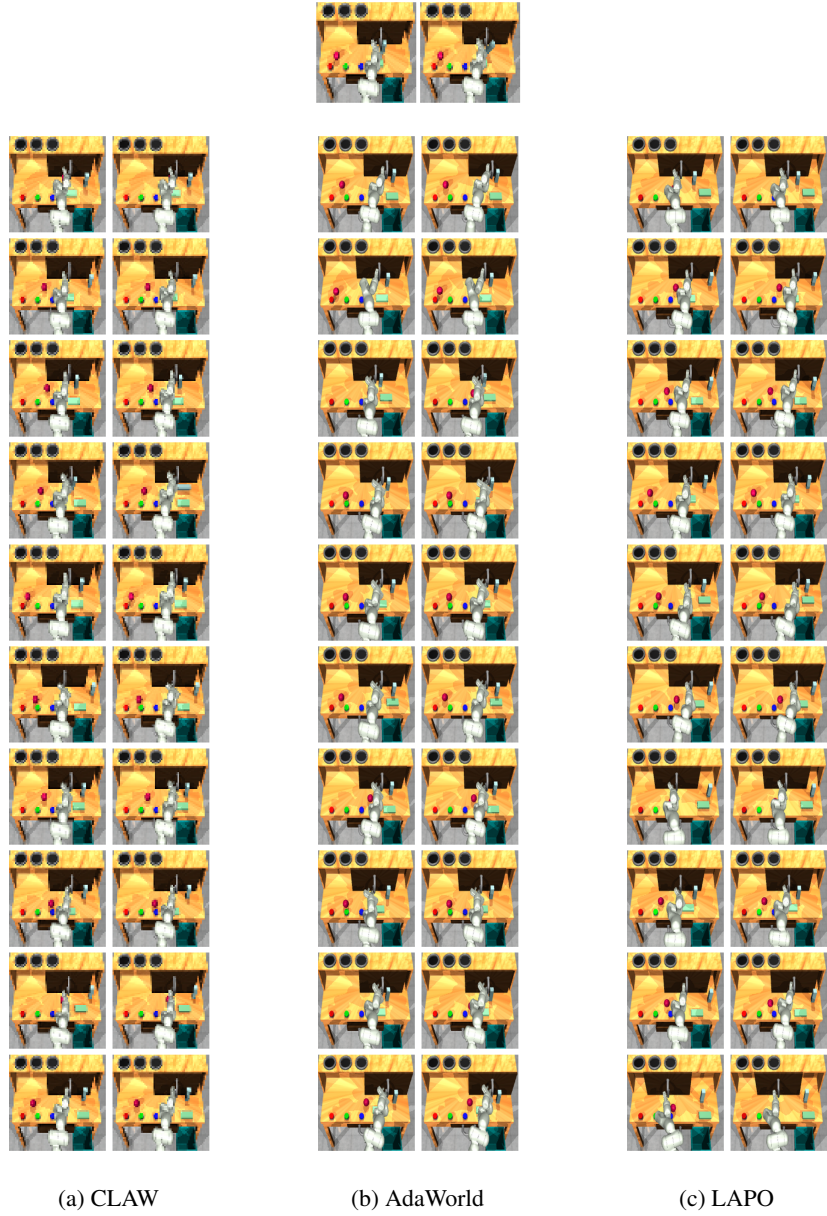


Figure 8: **Closest Latent Action retrieval via L_2 nearest-neighbor:** (Top) query transition. (Bottom) top-10 neighbors retrieved via L_2 search in each method’s latent space (CLAW, AdaWorld, LAPO). Retrieval is independent per encoder. All frames shown as grayscale with optical flow ($t \rightarrow t+1$) overlaid (Middlebury color wheel).



(a) CLAW

(b) AdaWorld

(c) LAPO

Figure 9: **Closest Latent Action retrieval via L_2 nearest-neighbor:** (Top) query transition. (Bottom) top-10 neighbors retrieved via L_2 search in each method's latent space (CLAW, AdaWorld, LAPO). Retrieval is independent per encoder.

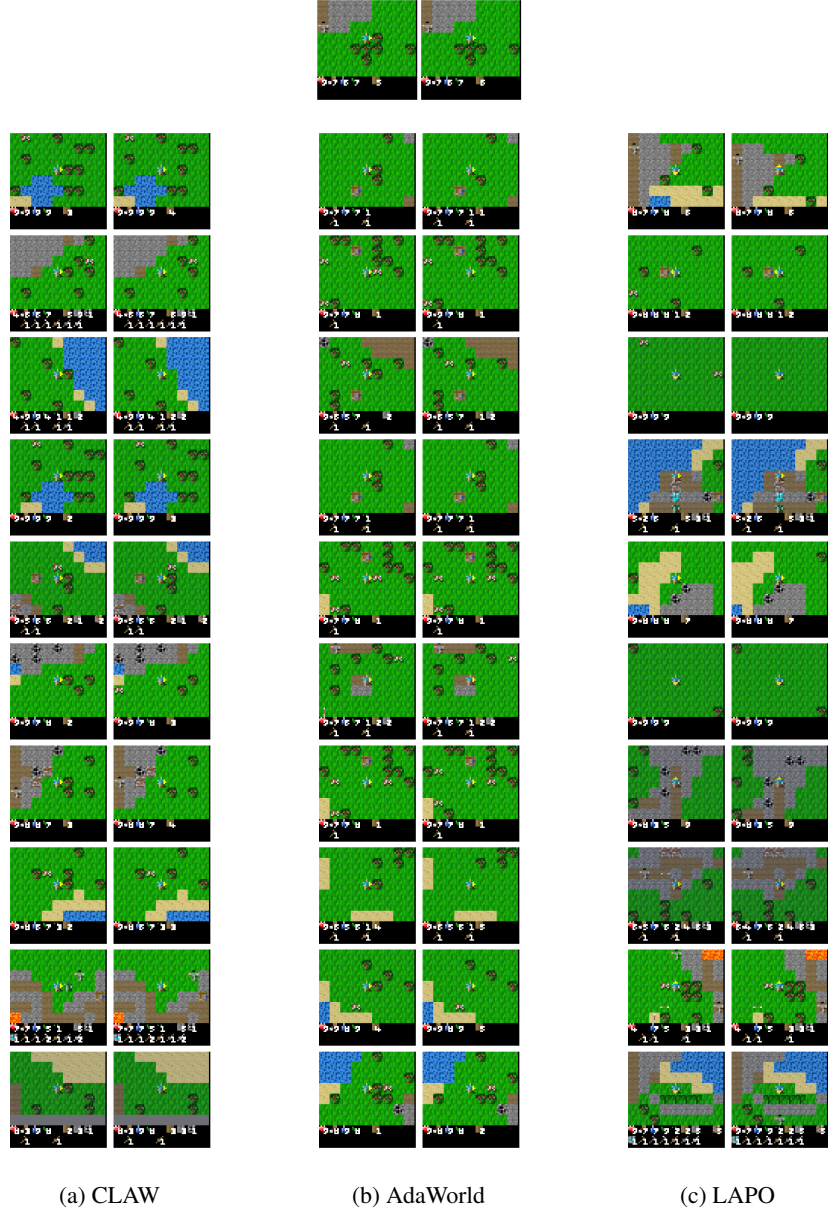


Figure 10: **Closest Latent Action retrieval via L_2 nearest-neighbor:** (Top) query transition. (Bottom) top-10 neighbors retrieved via L_2 search in each method’s latent space (CLAW, AdaWorld, LAPO). Retrieval is independent per encoder.

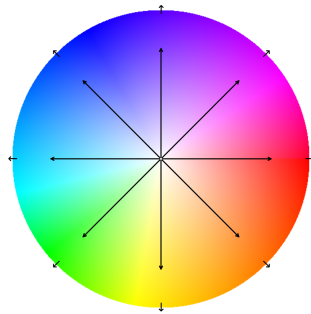


Figure 11: Middlebury Color Wheel.

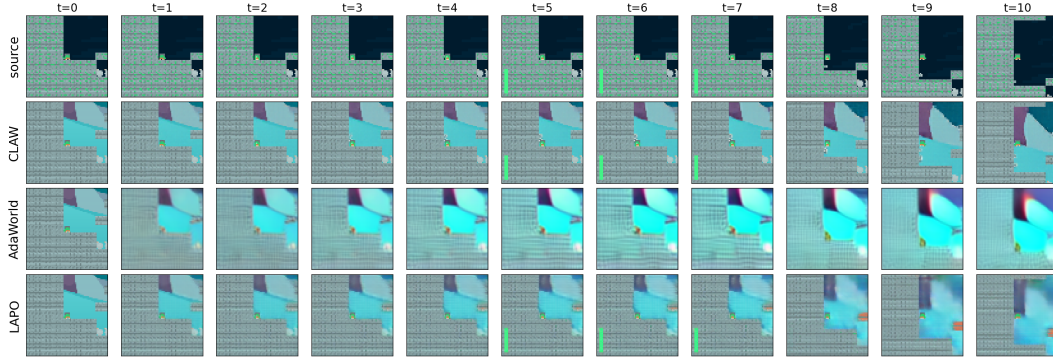


Figure 12: Action transfer on Procgen Ninja. Top: source trajectory. Below: rollouts on the target initial frame using CLAW, AdaWorld, and LAPO.

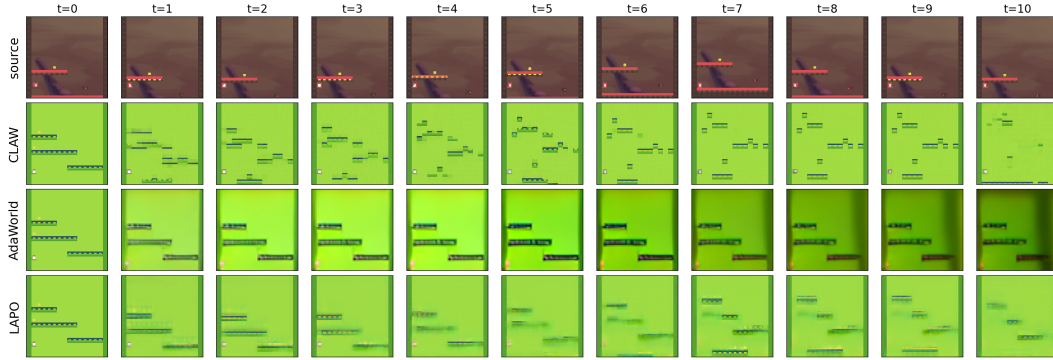


Figure 13: Action transfer on Procgen Climber. Top: source trajectory. Below: rollouts on the target initial frame using CLAW, AdaWorld, and LAPO.

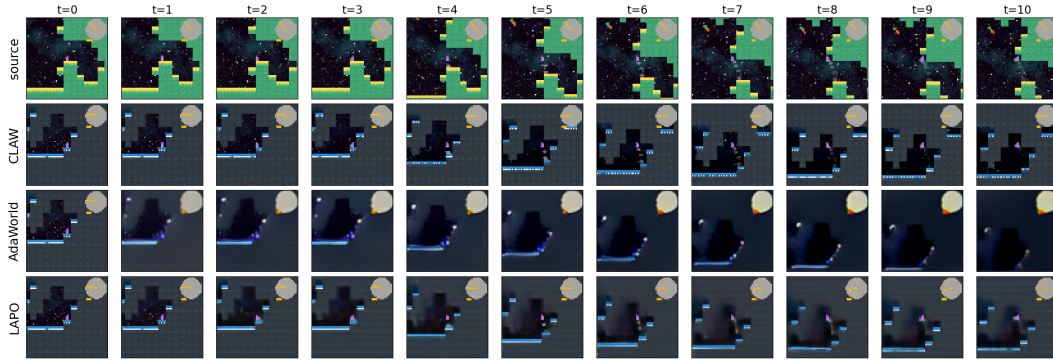


Figure 14: Action transfer on Procgen Jumper. Top: source trajectory. Below: rollouts on the target initial frame using CLAW, AdaWorld, and LAPO.

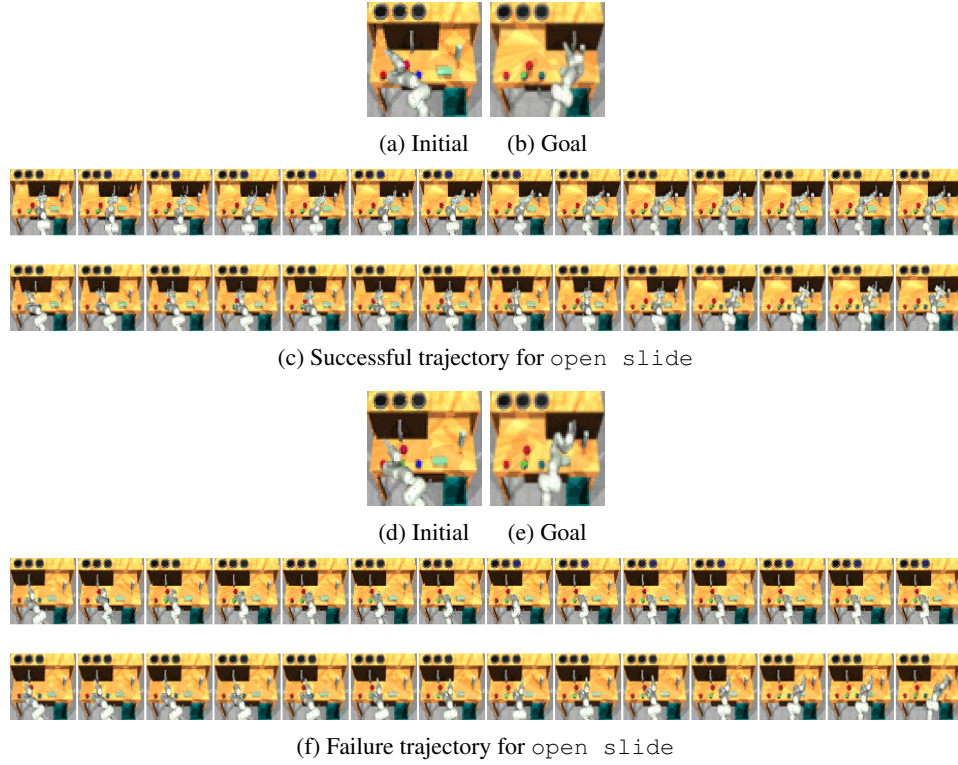


Figure 15: Visual planning results in the **VP2** benchmark using the **robodesk** environment for the task `open_slide`. We show both successful and failure trajectories generated by the planner.

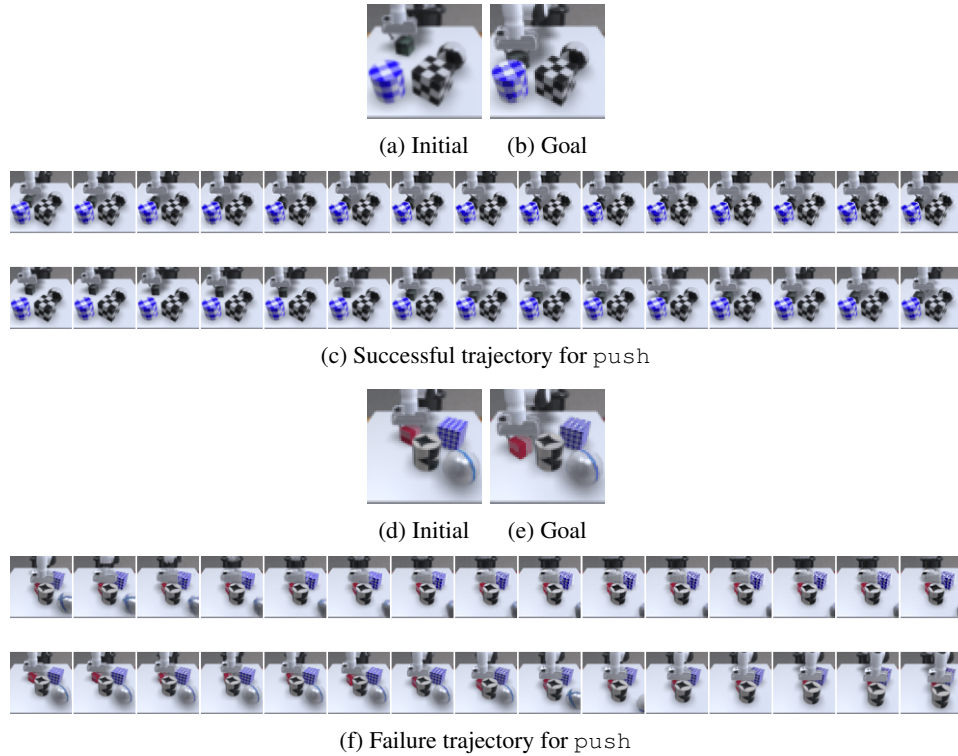


Figure 16: Visual planning results in the **Robosuite** benchmark using the **robosuite** environment for the task `push`. We show both successful and failure trajectories generated by the planner.

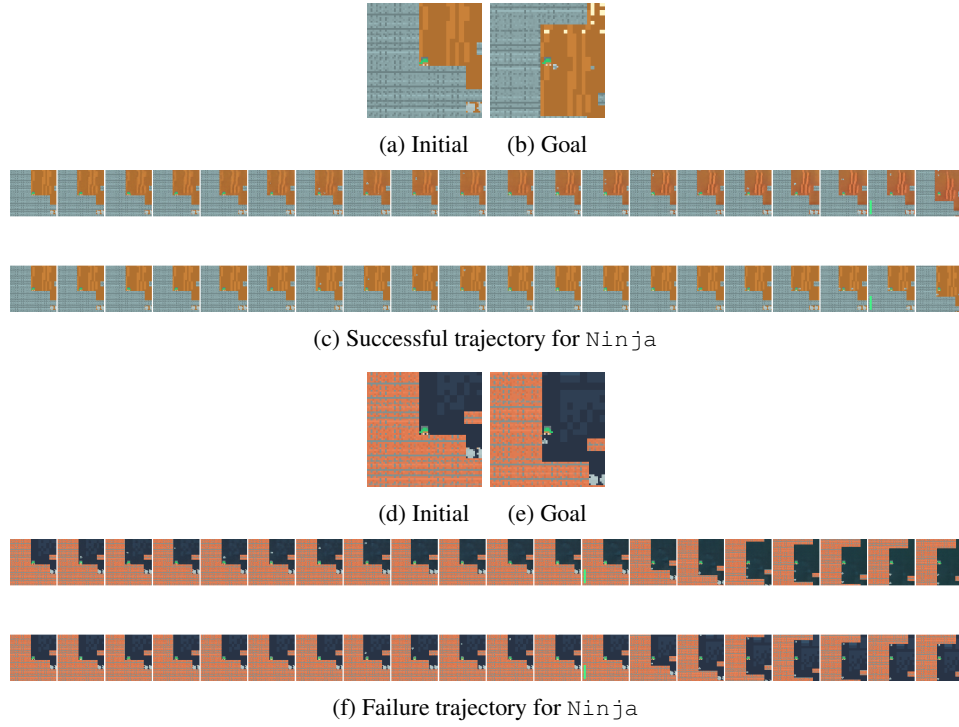


Figure 17: Visual planning results in the **ProcGen** benchmark using the **ninja** environment for the task **Ninja**. We show both successful and failure trajectories generated by the planner.

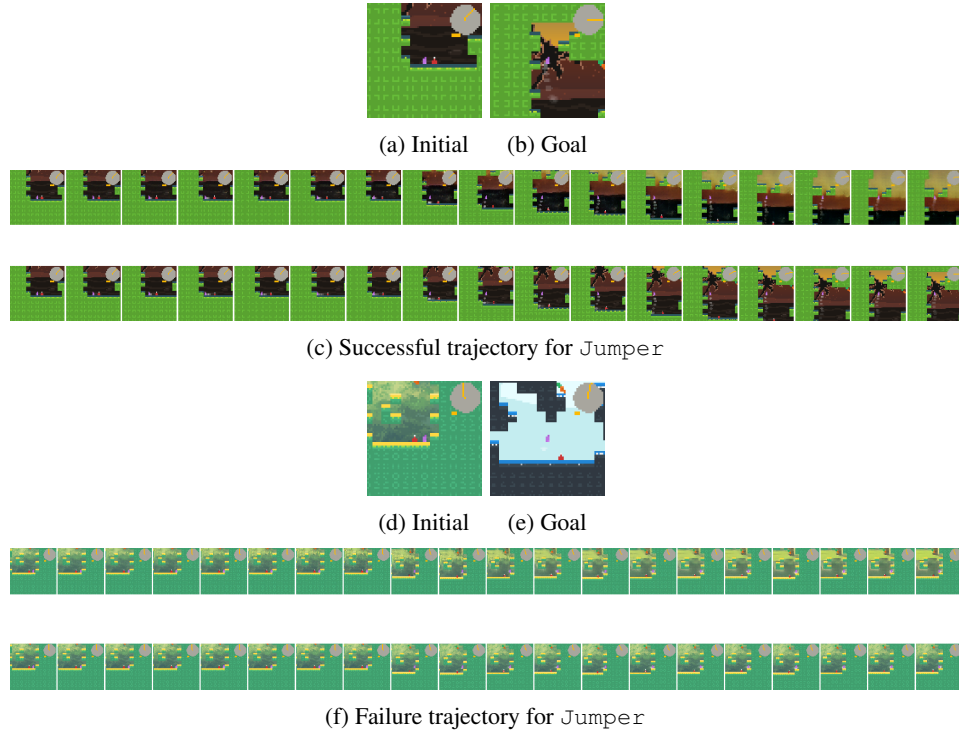


Figure 18: Visual planning results in the **ProcGen** benchmark using the **jumper** environment for the task **Jumper**. We show both successful and failure trajectories generated by the planner.